SE423 Lab 06: Behavioral Design Patterns

1. Strategy Pattern

The Strategy pattern encapsulates a family of algorithms separately, allowing them to be selected and interchanged at runtime without modifying the class that uses them.

Scenario: A food delivery application calculates delivery charges based on the distance and the selected delivery method. Customers can choose between standard and express delivery, and the application allows them to switch between delivery methods at runtime.

Activity:

1. Create a **DeliveryStrategy** interface with a method that calculates the delivery fee based on distance.
2. Implement two delivery strategies:
    - StandardDelivery
    - ExpressDelivery
3. Create a **DeliveryService** class that accepts a delivery strategy and calculates the delivery fee using the selected strategy.
4. Allow the delivery strategy to be changed at runtime using the **setStrategy()** method.
5. Test both strategies for a delivery distance of 5 units and display the calculated delivery fees.

Implementation:

Step 1: Create the strategy interface.

```
interface DeliveryStrategy {
   double calculateFee(double distance);
}
```

Step 2: Implement different strategies.

```
class StandardDelivery implements DeliveryStrategy {
   public double calculateFee(double distance) {
      return distance * 20;
   }
}
class ExpressDelivery implements DeliveryStrategy {
   public double calculateFee(double distance) {
      return distance * 40 + 100;
    }
}
```

Step 3: Create a class that uses the strategy.

```
class DeliveryService {
   private DeliveryStrategy strategy;
   public void setStrategy(DeliveryStrategy strategy) {
      this.strategy = strategy; }
   public double getFee(double distance) {
      return strategy.calculateFee(distance);
} }
```

Step 4: Test the implementation.

```
public class Main {
   public static void main(String[] args) {
      DeliveryService service = new DeliveryService();
      service.setStrategy(new StandardDelivery());
      System.out.println(service.getFee(5));
      service.setStrategy(new ExpressDelivery());
      System.out.println(service.getFee(5));
   }
}
```

Practice: Implement a **PremiumDelivery** strategy that charges a fixed fee of Rs. 150 regardless of distance. Add this strategy without modifying the existing **DeliveryService** class.

2. Observer Pattern

The Observer pattern establishes a one-to-many relationship between objects, allowing registered observers to be notified automatically when the subject's state changes or an event occurs.

Scenario: A food delivery application needs to notify the customer and delivery rider whenever an order's status changes. For example, when the order enters the preparation stage or goes out for delivery, both parties should receive an updated status.

Activity:

1. Create an **OrderObserver** interface with an **update()** method that accepts the order status.
2. Implement two observers:
   - Customer
   - DeliveryRider
3. Create an Order class that maintains a list of registered observers and provides a method to add observers.
4. Implement a **setStatus()** method that updates the order status and notifies all registered observers.
5. Create an order, register the customer and delivery rider, and test notifications for the "**Preparing**" and "**Out for delivery**" statuses.

Implementation:

Step 1: Create the observer interface.

```
interface OrderObserver {
   void update(String status);
}
```

Step 2: Create concrete observers.

```
class Customer implements OrderObserver {
   public void update(String status) {
      System.out.println("Customer notified: " + status);
   }}
class DeliveryRider implements OrderObserver {
   public void update(String status) {
      System.out.println("Rider notified: " + status);
  }
}
```

Step 3: Create the subject that manages observers.

```
import java.util.ArrayList;
import java.util.List;
class Order {
  private List<OrderObserver> observers = new ArrayList<>();
  public void addObserver(OrderObserver observer) {
    observers.add(observer);
  }
  public void setStatus(String status) {
    System.out.println("Order status: " + status);
    for (OrderObserver observer : observers) {
      observer.update(status);
    }}}
```

Step 4: Test the implementation.

```
public class Main {
  public static void main(String[] args) {
    Order order = new Order();
    order.addObserver(new Customer());
    order.addObserver(new DeliveryRider());
    order.setStatus("Preparing");
    order.setStatus("Out for delivery");
  }
}
```

Practice: Add a **RestaurantManager** observer and modify the program to support removing an observer so that the removed observer no longer receives status updates.

3. Command Pattern

The Command pattern encapsulates a request as an object, separating the requester from the object that executes it and enabling operations to be executed, queued, logged, or undone.

Scenario: A food delivery application allows restaurant staff to accept or cancel customer orders. Each operation is encapsulated as a separate command and executed through a restaurant controller.

Activity:

1. Create a Command interface containing an execute() method.
2. Create a **FoodOrder** class with methods to accept and cancel an order.
3. Implement two commands: **AcceptOrderCommand** and **CancelOrderCommand**, each invoking the corresponding operation on a **FoodOrder** object.
4. Create a **RestaurantController** class with an **executeCommand()** method that executes the received command.
5. Create a food order and execute both commands through the same controller.
6. Verify that each command invokes the appropriate operation without placing order-specific logic inside the controller.

Implementation:

Step 1: Create the command interface.

```
interface Command {
    void execute();
}
```

Step 2: Create the receiver.

```
class FoodOrder {
    public void accept() {
        System.out.println("Order accepted.");
    }
    public void cancel() {
        System.out.println("Order cancelled.");
    } }
```

Step 3: Implement the commands.

```
class AcceptOrderCommand implements Command {
    private FoodOrder order;
    public AcceptOrderCommand(FoodOrder order) {
        this.order = order;
    }
    public void execute() {
        order.accept();
    } }
class CancelOrderCommand implements Command {
    private FoodOrder order;
    public CancelOrderCommand(FoodOrder order) {
        this.order = order;
    }
    public void execute() {
        order.cancel();
    } }
```

Step 4: Create the invoker.

```
class RestaurantController {
    public void executeCommand(Command command) {
        command.execute();
    } }
```

Step 5: Test the implementation.

```
public class Main {
    public static void main(String[] args) {
        FoodOrder order = new FoodOrder();
        RestaurantController controller = new RestaurantController();
        controller.executeCommand(new AcceptOrderCommand(order));
        controller.executeCommand(new CancelOrderCommand(order));
    } }
```

Practice: Implement a **PrepareOrderCommand** that invokes a preparation operation on **FoodOrder**. Execute it through the existing **RestaurantController** without modifying the controller class.

4. Template Method Pattern

The Template Method pattern defines an algorithm's structure in a base class while allowing subclasses to customize specific steps without changing the overall sequence.

Scenario: A food delivery application processes payments using different methods, such as cash and card. Each payment follows the same sequence of validating the order, processing the payment, and generating a receipt, while the payment processing step varies by method.

Activity:

1. Create an abstract **PaymentProcessor** class.
2. Implement a **processPayment()** template method that defines the payment processing sequence.
3. Add common methods to validate the order and generate a receipt.
4. Define an abstract **makePayment()** method for payment-specific processing.
5. Implement **CashPayment** and **CardPayment** subclasses with their respective payment messages.
6. Test both implementations and verify that they follow the same processing sequence while performing payment differently.

Implementation:

Step 1: Create the abstract class.

```
abstract class PaymentProcessor {
    public final void processPayment() {
        validateOrder();
        makePayment();
        generateReceipt();
    }
    private void validateOrder() {
        System.out.println("Order validated.");
    }
    protected abstract void makePayment();
    private void generateReceipt() {
        System.out.println("Receipt generated.");
    }
}
```

Step 2: Implement payment methods.

```
class CashPayment extends PaymentProcessor {
    protected void makePayment() {
        System.out.println("Payment received in cash.");
    }
}
class CardPayment extends PaymentProcessor {
    protected void makePayment() {
        System.out.println("Payment processed by card.");
    }
}
```

Step 3: Test the implementation.

```
public class Main {
   public static void main(String[] args) {
      PaymentProcessor cash = new CashPayment();
      cash.processPayment();
      System.out.println("---");
      PaymentProcessor card = new CardPayment();
      card.processPayment();
   }
}
```

Practice: Implement a **WalletPayment** subclass that provides its own payment message while following the existing processing sequence without modifying the **PaymentProcessor** class.

5. State Pattern

The State pattern allows an object's behavior to change dynamically based on its internal state.

Scenario: A food delivery application manages an order through three states: placed, prepared, and delivered. The order transitions to the next state when an operation is performed, while further transitions are prevented once the order has been delivered.

Activity:

1. Create an **OrderState** interface with **next()** and **getStatus()** methods.
2. Implement three state classes**: PlacedState**, **PreparingState**, and **DeliveredState**.
3. Define the appropriate state transition in each class using the **OrderContext** object.
4. Create an **OrderContext** class that maintains the current state and provides methods to change and display it.
5. Test the order's progression from placed to prepare and then delivered, displaying the status after each transition.
6. Attempt another transition after delivery and verify that the application displays a message indicating that the order has already been delivered.

Implementation:

Step 1: Create the state interface.

```
interface OrderState {
   void next(OrderContext order);
   String getStatus();
}
```

Step 2: Implement individual states.

```
class PlacedState implements OrderState {
   public void next(OrderContext order) {
      order.setState(new PreparingState());
   }
   public String getStatus() {
      return "Order placed"; } }
class PreparingState implements OrderState {
   public void next(OrderContext order) {
      order.setState(new DeliveredState());
   }
```

```
    public String getStatus() {
        return "Order preparing";
    }
}
class DeliveredState implements OrderState {
    public void next(OrderContext order) {
        System.out.println("Order is already delivered.");
    }
    public String getStatus() {
        return "Order delivered";
    }
}
```

Step 3: Create the context.

```
class OrderContext {
    private OrderState state = new PlacedState();
    public void setState(OrderState state) {
        this.state = state;
    }
    public void nextState() {
        state.next(this);
    }
    public void showStatus() {
        System.out.println(state.getStatus());
    }}
```

Step 4: Test the implementation.

```
public class Main {
    public static void main(String[] args) {
        OrderContext order = new OrderContext();
        order.showStatus();
        order.nextState();
        order.showStatus();
        order.nextState();
        order.showStatus();
        order.nextState();
    }}
```

Practice: Add a **ReadyForPickupState** between **PreparingState** and **DeliveredState**. Update the transitions so that the order follows the correct sequence without skipping the new state.

6. Iterator Pattern

The Iterator pattern enables sequential access to collection elements without exposing the collection's internal structure.

Scenario: A food delivery application maintains a restaurant menu containing different food items. The application must display all menu items sequentially without exposing the internal structure of the collection.

Activity:

1. Create a **MenuItem** class that stores the name of a food item and provides a method to retrieve it.
2. Create a **RestaurantMenu** class that maintains a collection of menu items and provides a method to add items.
3. Implement the **Iterable<MenuItem>** interface in **RestaurantMenu** and provide an iterator for traversing the collection.
4. Add three menu items and display their names using an enhanced loop.

Implementation:

Step 1: Create the menu item class.

```
class MenuItem {
   private String name;
   public MenuItem(String name) {
      this.name = name; }
   public String getName() {
      return name;
   } }
```

Step 2: Create the collection and iterator.

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
class RestaurantMenu implements Iterable<MenuItem> {
   private List<MenuItem> items = new ArrayList<>();
   public void addItem(MenuItem item) {
      items.add(item);  }
   public Iterator<MenuItem> iterator() {
      return items.iterator();
   } }
```

Step 3: Test the implementation.

```
public class Main {
   public static void main(String[] args) {
      RestaurantMenu menu = new RestaurantMenu();
      menu.addItem(new MenuItem("Burger"));
      menu.addItem(new MenuItem("Pizza"));
      menu.addItem(new MenuItem("Sandwich"));
      for (MenuItem item : menu) {
         System.out.println(item.getName());
      }
   }
}
```

Practice: Add five menu items and display them using iteration. Change the internal collection from **ArrayList** to **LinkedList** and verify that the traversal code works without modification.

Decoupling Algorithms and Control Flow:
Understand how to separate an algorithm's implementation from the code that controls when and how it is executed.

Scenario: A food delivery application calculates delivery fees using different delivery methods. The application should allow customers to switch delivery methods without modifying the code responsible for processing a delivery request.

Step 1: Create a delivery fee algorithm interface

```
interface DeliveryStrategy {
    double calculateFee(double distance);
}
```

Step 2: Implement different algorithms

```
class StandardDelivery implements DeliveryStrategy {
    public double calculateFee(double distance) {
        return distance * 20;
    } }
class ExpressDelivery implements DeliveryStrategy {
    public double calculateFee(double distance) {
        return distance * 40 + 100;
    } }
```

Step 3: Create the class that controls the delivery process

```
class DeliveryService {
    public void processDelivery(
            DeliveryStrategy strategy, double distance) {
        double fee = strategy.calculateFee(distance);
        System.out.println("Delivery distance: " + distance);
        System.out.println("Delivery fee: " + fee);
    } }
```

Step 4: Test the implementation

```
public class Main {
    public static void main(String[] args) {
        DeliveryService service = new DeliveryService();
        service.processDelivery(new StandardDelivery(), 5);
        System.out.println("---");
        service.processDelivery(new ExpressDelivery(), 5);
    } }
```

Practice: Add a **PremiumDelivery** class that implements **DeliveryStrategy** and calculates the fee using a different algorithm. Execute it through the existing **DeliveryService** class without modifying that class.

Discussion questions

1. Which class contains the delivery fee algorithms?
2. Which class controls the delivery processing flow?
3. Why can a new delivery algorithm be added without modifying DeliveryService?
4. What would need to change if all delivery algorithms were implemented using if-else statements inside DeliveryService?